

深圳大学实验报告

课程名称：____互联网编程____

实验项目名称：____熟悉互联网编程开发环境____

学院：____计算机与软件学院____

专业：____计算机科学与技术____

指导教师：____李梦柯____

报告人：____姚泽棉____学号：2024150026 班级：____高性能班____

实验时间：____2026/03/12____

实验报告提交时间：____2026/03/13____

教务处制

一、实验目的与内容：

目的：掌握面向 java 的互联网编程开发环境的搭建。学习、掌握 java 网络程序编写基本步骤，例如，基于 MyEclipse 或者 IntelliJ IDEA 平台编写简单的基于 TCP 的 多线程文件收发程序，掌握编译、运行等基本步骤和操作。

内容要求：

1. 搭建开发环境（基本要求，50 分）：

（在自己电脑上）下载安装和配置 Java 开发环境（如 Eclipse 或 IntelliJ IDEA 等），并熟悉该开发环境中的常用操作。

请在报告中给出搭建环境成功的截图和简要文字说明，简述自己所了解掌握的常用操作。

2. 尝试编写一个基于 TCP 的多线程文件收发程序（CS 架构）（50 分）：

可试着根据 java 编程课所学到的 java socket 编程技术，尝试编写一个基于 TCP 的多线程文件收发程序，需满足：

1. 服务端程序预先定义好需要发送的文件并等待客户端的连接。

2. 客户端连接成功后，服务端将文件发送给客户端，客户端将文件保存到本地。需要在同一个 TCP 连接内发送多个文件，操作系统支持的前提下，不限制文件的类型和大小。

注意：

1. 实验报告中需要有实验结果的截图。

二、实验过程和代码与结果

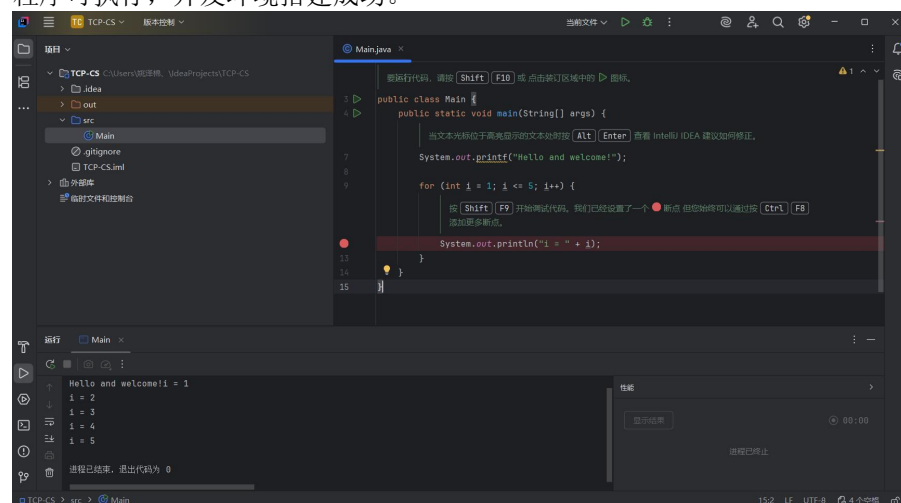
1. 开发环境的搭建实验过程及结果

查看在 CMD 输入中 java -version 和 javac -version 的执行结果，显示 JDK 1.8 版本信息，证明 JDK 配置成功；

```
C:\Users\姚泽棉>java -version
java version "21.0.8" 2025-07-15 LTS
Java(TM) SE Runtime Environment (build 21.0.8+12-LTS-250)
Java HotSpot(TM) 64-Bit Server VM (build 21.0.8+12-LTS-250, mixed mode, sharing)

C:\Users\姚泽棉>javac -version
javac 21.0.8
```

程序可执行，开发环境搭建成功。



2. 给出满足内容要求 2 的程序源码及运行结果，简述思路或实验过程。

程序源码：

服务端

```
1 import java.io.*;
2 import java.net.*;
3
4 public class TCPServer {
5
6     // 预先定义好需要发送的文件路径（可自行修改）
7     private static final String[] FILES_TO_SEND = { 4 个用法
8         "src/1.txt",
9         "src/2.jpg",
10        "src/3.pdf"
11    };
12
13    private static final int PORT = 9999; 3 个用法
14
15    public static void main(String[] args) {
16        System.out.println("=== TCP 文件发送服务端 ===");
17
18        // 校验待发送文件是否存在
19        for (String filePath : FILES_TO_SEND) {
20            File f = new File(filePath);
21            if (!f.exists() || !f.isFile()) {
22                System.err.println("[错误] 文件不存在: " + f.getAbsolutePath());
23                System.err.println("请将待发送文件放到程序运行目录下，或修改 FILES_TO_SEND 路径。");
24                return;
25            }
26        }
27
28        ServerSocket serverSocket = null;
29        Socket clientSocket = null;
30        DataOutputStream dos = null;
31        BufferedInputStream bis = null;
32
33        try {
34            serverSocket = new ServerSocket(PORT);
35            System.out.println("[服务端] 正在监听端口 " + PORT + "，等待客户端连接...");
36
37            clientSocket = serverSocket.accept();
38            System.out.println("[服务端] 客户端已连接: " + clientSocket.getInetAddress());
39
40            dos = new DataOutputStream(clientSocket.getOutputStream());
41
42            // 1. 先发送文件数量
43            dos.writeInt(FILES_TO_SEND.length);
44            dos.flush();
45            System.out.println("[服务端] 即将发送 " + FILES_TO_SEND.length + " 个文件");
46
47            // 2. 逐个发送文件
48            byte[] buffer = new byte[8192];
49            for (String filePath : FILES_TO_SEND) {
50                File file = new File(filePath);
51
52                // 发送文件名（UTF-8 编码）
53                dos.writeUTF(file.getName());
54                // 发送文件大小
55                dos.writeLong(file.length());
56                dos.flush();
```

```

58         // 发送文件内容
59         bis = new BufferedInputStream(new FileInputStream(file));
60         long totalSent = 0;
61         int bytesRead;
62         while ((bytesRead = bis.read(buffer)) != -1) {
63             dos.write(buffer, 0, bytesRead);
64             totalSent += bytesRead;
65         }
66         dos.flush();
67         bis.close();
68         bis = null;
69
70         System.out.println("[服务端] 文件发送完成: " + file.getName()
71             + " (" + totalSent + " 字节)");
72     }
73
74     System.out.println("[服务端] 所有文件发送完毕!");
75
76     } catch (BindException e) {
77         System.err.println("[错误] 端口 " + PORT + " 已被占用, 请更换端口或关闭占用程序。");
78     } catch (IOException e) {
79         System.err.println("[错误] 传输过程中发生IO异常: " + e.getMessage());
80         e.printStackTrace();
81     } finally {
82         // 关闭资源
83         try { if (bis != null) bis.close(); } catch (IOException ignored) {}
84         try { if (dos != null) dos.close(); } catch (IOException ignored) {}
85         try { if (clientSocket != null) clientSocket.close(); } catch (IOException ignored) {}
86         try { if (serverSocket != null) serverSocket.close(); } catch (IOException ignored) {}
87         System.out.println("[服务端] 资源已释放, 程序结束。");
88     }
89 }

```

客户端

```

1  import java.io.*;
2  import java.net.*;
3
4  public class TCPClient {
5
6      private static final String SERVER_HOST = "127.0.0.1"; // 2个用法
7      private static final int SERVER_PORT = 9999; // 2个用法
8      private static final String SAVE_DIR = "received_files"; // 接收文件的保存目录 1个用法
9
10     public static void main(String[] args) {
11         System.out.println("=== TCP 文件接收客户端 ===");
12
13         // 创建保存目录
14         File saveDir = new File(SAVE_DIR);
15         if (!saveDir.exists()) {
16             saveDir.mkdirs();
17         }
18
19         Socket socket = null;
20         DataInputStream dis = null;
21         BufferedOutputStream bos = null;
22
23         try {
24             System.out.println("[客户端] 正在连接服务器 " + SERVER_HOST + ":" + SERVER_PORT + " ...");
25             socket = new Socket(SERVER_HOST, SERVER_PORT);
26             System.out.println("[客户端] 连接成功!");
27
28             dis = new DataInputStream(socket.getInputStream());

```

```

30 // 1. 读取文件数量
31 int fileCount = dis.readInt();
32 System.out.println("[客户端] 服务端将发送 " + fileCount + " 个文件");
33
34 // 2. 逐个接收文件
35 byte[] buffer = new byte[8192];
36 for (int i = 0; i < fileCount; i++) {
37     // 读取文件名
38     String fileName = dis.readUTF();
39     // 读取文件大小
40     long fileSize = dis.readLong();
41
42     System.out.println("[客户端] 正在接收第 " + (i + 1) + " 个文件: "
43         + fileName + " (" + fileSize + " 字节)");
44     File outFile = new File(saveDir, fileName);
45     bos = new BufferedOutputStream(new FileOutputStream(outFile));
46
47     long remaining = fileSize;
48     int bytesRead;
49     while (remaining > 0) {
50         int toRead = (int) Math.min(buffer.length, remaining);
51         bytesRead = dis.read(buffer, off, 0, toRead);
52         if (bytesRead == -1) {
53             throw new IOException("连接意外断开, 文件 " + fileName + " 接收不完整");
54         }
55         bos.write(buffer, off, 0, bytesRead);
56         remaining -= bytesRead;
57     }
58     bos.flush();
59     bos.close();
60     bos = null;
61
62     System.out.println("[客户端] 文件保存成功: " + outFile.getAbsolutePath());
63 }
64
65 System.out.println("[客户端] 所有文件接收完毕!");
66
67 } catch (ConnectException e) {
68     System.err.println("[错误] 无法连接到服务器, 请确认服务端已启动且端口正确。");
69 } catch (IOException e) {
70     System.err.println("[错误] 传输过程中发生IO异常: " + e.getMessage());
71     e.printStackTrace();
72 } finally {
73     try { if (bos != null) bos.close(); } catch (IOException ignored) {}
74     try { if (dis != null) dis.close(); } catch (IOException ignored) {}
75     try { if (socket != null) socket.close(); } catch (IOException ignored) {}
76     System.out.println("[客户端] 资源已释放, 程序结束。");
77 }
78 }
79

```

运行结果

服务器端运行结果

```

public class TcpsServer {
    public static void main(String[] args) {
        // ... (code omitted) ...

        System.out.println("[服务端] 所有文件接收完毕");

        } catch (BindException e) {
            System.err.println("[错误] 端口 " + PORT + " 已被占用, 请更换端口或关闭占用程序。");
        } catch (IOException e) {
            System.err.println("[错误] 传输过程中发生IO异常: " + e.getMessage());
            e.printStackTrace();
        } finally {
            // ... (code omitted) ...
        }
    }
}

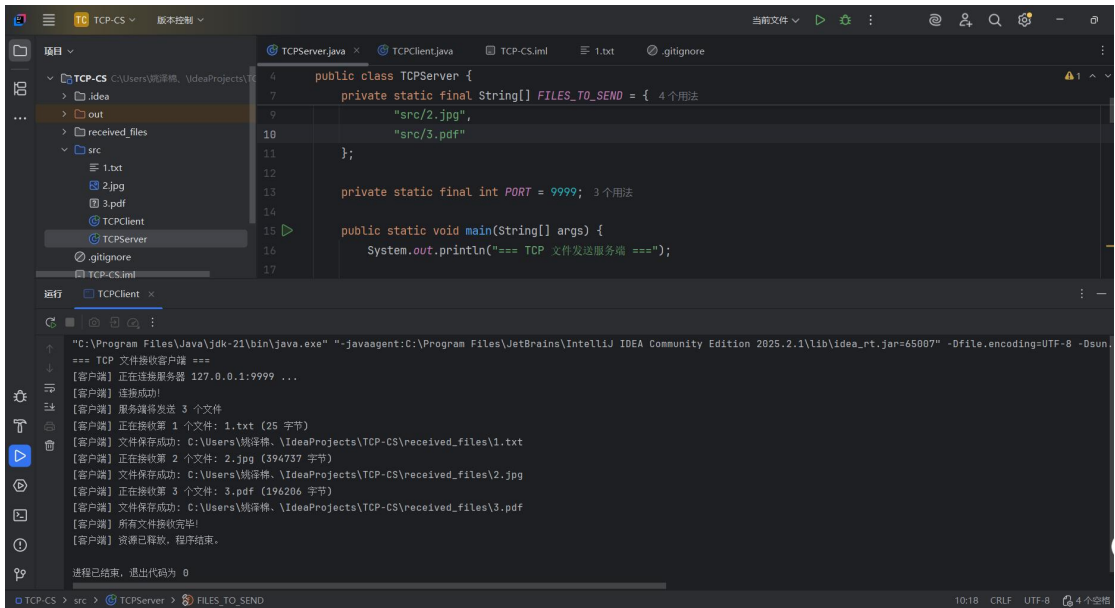
```

```

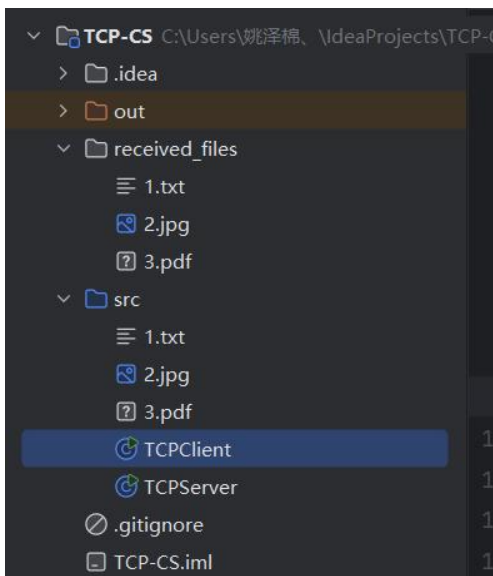
[服务端] 正在监听端口 9999, 等待客户端连接...
[服务端] 客户端已连接: 127.0.0.1
[服务端] 接收文件 1.txt
[服务端] 文件接收完成: 1.txt (25 字节)
[服务端] 文件接收完成: 2.jpg (104728 字节)
[服务端] 文件接收完成: 3.pdf (194286 字节)
[服务端] 所有文件接收完毕!
[服务端] 资源已释放, 程序结束。

```

客户端接受结果



文件接收结果



实验思路

本程序采用 C/S 架构, 基于 Java Socket API 实现 TCP 文件传输。为了在同一个 TCP 连接中传输多个文件, 设计了如下简单应用层协议:

1. 服务端先发送一个 int (4 字节), 表示待发送的文件数量;
2. 对于每个文件, 依次发送: 文件名 (writeUTF, 含长度前缀) → 文件大小 (long, 8 字节) → 文件内容 (原始字节流);
3. 客户端按照相同顺序解析, 根据文件大小精确读取对应字节数, 从而在一条 TCP 连接上区分出多个文件的边界。

实验步骤:

1. 在项目运行目录下准备若干测试文件（如 txt、jpg、pdf 各一个）；
2. 先运行 FileServer，服务端开始在端口 9999 监听；
3. 再运行 FileClient，客户端连接服务端后自动接收文件，保存到 received_files 目录；
4. 对比收发文件的大小与内容，验证传输正确性。

三、实验总结

（此处写你的过程，比如遇到的错误，以及解决方法，你的所想、所得。不要过度依赖大模型。）

1. 服务端 “文件不存在” 错误

- 问题现象：服务端启动时提示 “文件不存在”，但文件已放置在项目目录中，程序无法定位文件路径。
- 原因分析：文本文件要保存 ctrl+s，才会显示所占字节，否则显示 0 字节；文件名写成 1.txt，而 .txt 后缀被计算机隐藏，实际名字为 1.txt.txt。
- 解决方法：保存文件并将名字改成 1。

2. 客户端 UTFDataFormatException 格式异常

- 问题现象：第一个文件可正常接收，第二个文件接收时抛出 java.io.UTFDataFormatException: malformed input around byte 0，控制台显示 “实际接收字节数远大于预期”。
- 原因分析：1. 服务端使用缓冲输出流时，缓冲区刷新不及时，导致 “文件名 + 文件大小” 的元信息与文件字节数据混叠，破坏了 “文件数→文件名→文件大小→字节数据” 的通信协议；2. 客户端读取文件字节时未严格限制长度，多读了下一个文件的元信息数据，导致 readUTF() 解析文件名时读到非 UTF 格式的字节。
- 解决方法：1. 服务端在发送元信息（文件名、大小）和文件字节后均调用 dos.flush()，强制刷新缓冲区，确保数据按协议顺序发送；2. 客户端优化读取逻辑：通过 Math.min(buffer.length, remaining) 限制单次读取字节数，严格按文件大小接收数据，避免多读；3. 最终代码中新增 remaining 变量跟踪剩余待接收字节，循环读取直至 remaining=0，彻底杜绝数据错位。

总结

本次实验不仅掌握了 TCP 文件传输的核心逻辑，更体会到 “细节决定成败” —— 路径拼写、缓冲区刷新、协议顺序等微小偏差，都会导致程序运行失败。通过问题排查与代码优化，也建立了 “先定位问题根源，再针对性解决” 的编程思维，为后续互联网编程学习奠定了基础。

AI 使用说明

本人在完成本实验报告过程中，使用了大语言模型作为辅助工具。

使用工具名称：Claude-Opus-4.6（如 ChatGPT / DeepSeek / Qwen 等）

使用用途：

- 查阅资料
- 代码检查
- 报告语言优化

本人承诺：

实验报告中的代码与内容均已理解，并能够向指导老师解释程序实现原理。

学生姓名：姚泽楦

学号：2024150026

日期：2026/03/13

指导教师批阅意见：

成绩评定：

指导教师签字：

年 月 日

备注：

注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。

2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。